

Neighborhoods, Ripley's K, etc.

Gabriel Arellano

2026-05-26

Contents

1	Introduction	1
2	List of old and new functions	2
3	Functions	3
4	Examples	12
4.1	Example 1: Ripley's K and Besag's L	15
4.2	Example 2: Aggregation within species	37

1 Introduction

This is updated code to calculate abundances of neighbors at different scales, in circles and rings. It also contains instructions to compare observations with expectations, including Ripley's K and Besag's L.

Spatial neighborhood analyses are used to quantify how organisms, stems, species, functional groups, or other ecological units are arranged around focal individuals or locations. In forest ecology, these analyses are commonly used to evaluate local crowding, conspecific and heterospecific aggregation, spatial segregation, density dependence, habitat association, biomass accumulation around focal trees, and departures from spatial null expectations. Classical summaries include neighbor counts or densities within circles or rings, Ripley's K, and transformations such as Besag's L.

This tutorial introduces updated code and instructions for calculating neighborhood summaries at multiple spatial scales. The workflow separates several operations that were previously bundled together: identifying neighbors within a distance threshold, defining focal and reference

classes, handling circles and annuli, estimating available neighborhood area near valid boundaries (of arbitrary shape, but typically the rectangular plot limits), summarizing abundance as counts of points or other metrics, and comparing observations with null expectations. This separation is intentional.

Relative to the previous CTFS functions, the updated code is designed to be less tied to specific assumptions about rectangular plots, point classes, or abundance metrics. An important design criterion is to leave outside the functions everything that may require high flexibility by the potential users:

- filters / subsetting routines
- aggregating or summarizing functions
- null expectations

Below, there are examples on how to implement these things in a script.

Edge effects can be important. Handling boundaries and calculating areas is *way* more difficult than it seems. Much of the code below was developed to handle irregular boundaries, thus make the code as general as possible. It does not require rectangular boundaries and it does not use analytical corrections for edge effects. It directly uses a dense grid that count “points inside” to estimate the areas of irregular shapes (neighborhoods near possibly irregular boundaries). The density of the grid is adaptive, depending on the error tolerance. The functions include useful defaults, so you should be fine if you have a regular plot and just pass {x, y} coordinates of all the trees.

The code takes responsibility for the boundary- and area-related difficult calculations and for the *very fast* identification of pairs of neighbors. Filtering and summarizing (e.g. counting neighbors, calculating crowding indexes, etc.) will be responsibility of the user. Below there are useful templates or instructions.

The approach taken is very fast but demands a lot of memory, so this can be a practical limitation for large datasets. For the moment, I will wait for users’ feedback.

2 List of old and new functions

The code here substitutes these original CTFS functions:

- “CalcRingArea” in `spatial > RipUvK.r`
- “partialcirclearea” in `spatial > RipUvK.r`
- “circlearea” in `spatial > RipUvK.r`
- “Annuli” in `spatial > RipUvK.r`
- “NDcount” in `spatial > NeighborDensityFun.r`
- “NeighborDensities” in `spatial > NeighborDensityFun.r`
- “RipUvK” in `spatial > RipUvK.r`

The new functions are:

- `fast_distances_up_to_r`: the workhorse to calculate distances from one set of focal points (query) to their neighbors (taken from a reference dataset), within a given threshold. This takes advantage of the very fast `dbscan::frNN` function, and will be called often from within other functions here.
- `core_vs_edge`: an auxiliary function that classifies a set of points into “out” (outside the provided boundary of arbitrary shape), “edge” (inside the boundary, but closer than a given distance to it), and “core” (inside the boundary and far from it). The main goal of this auxiliary function is to save computational costs by avoiding numerical calculations for the area of full circles around “core” points.
- ...
- ...

Filters, abundance metrics, null expectations, and classes to compare, are all outside these functions.

3 Functions

The main function here is a function to calculate, fast, distances up to certain threshold. This is useful for the distances themselves and to identify neighbors.

```
# Fast calculation of distances up to a given radius:
fast_distances_up_to_r <- function(query, reference = NULL, radius, approx = 0) {

  # Parameters:
  # query: the set of focal points, around which we are going to look
  # reference: the set of potential neighbors; will default to be the same as "query"
  # radius: the size of the local neighborhood around the focal point
  # approx: precision parameter to be passed to dbscan::frNN

  # Reshape input
  if(is.null(reference)) reference <- query
  query <- as.matrix(query)
  reference <- as.matrix(reference)

  storage.mode(query) <- "double"
  storage.mode(reference) <- "double"

  # Calculate distances
  nn <- dbscan::frNN(
    x = reference,
```

```

    query = query,
    eps = radius,
    sort = FALSE,
    approx = approx
  )

  # Reorganize output
  k <- lengths(nn$id)

  data.table::data.table(
    query_i = rep(seq_along(k), k),
    reference_j = unlist(nn$id, use.names = FALSE),
    dist = unlist(nn$dist, use.names = FALSE)
  )[dist <= radius]
}

```

Before other functions, let's talk about boundaries. Edge effects are important: the available area for neighborhoods is smaller near the boundary than far from it. Here we try to make the code as general as possible, and a major improvement would be to allow boundaries of arbitrary shape. The most common case, however, would be the rectangular boundary. In the updated functions, the user can define boundaries as:

- sfc polygon or multipolygon object, for boundaries of any shape. Typical parameter name: "boundary"
- as rectangular windows, expressed as a vector of (xmin, ymin, xmax, ymax) plot limits. Typical parameter name: "window"

If the user is not explicit, the functions will guess the boundary by creating a complex boundary around the set of {x, y} points. It can be handy if one has trees classified into habitats and want to restrict the analyses to within particular habitats, for example. This is a proper default but it will take time to compute, so in general it will always be better to know and define the boundary beforehand, before entering costly loops, etc.

```

get_boundary <- function(boundary = NULL, window = NULL, xy = NULL, q = 5)
{
  # Minimal for spatial objects
  if (!is.null(boundary)) {
    if (inherits(boundary, "sfg")) {
      boundary <- sf::st_sfc(boundary)
    } else {
      boundary <- sf::st_geometry(boundary)
    }
  }
}

```

```

# From "window" to proper sf object
if (is.null(boundary) && !is.null(window)) {
  xmin <- window[1]
  ymin <- window[2]
  xmax <- window[3]
  ymax <- window[4]

  boundary <- sf::st_sfc(
    sf::st_polygon(list(rbind(
      c(xmin, ymin),
      c(xmax, ymin),
      c(xmax, ymax),
      c(xmin, ymax),
      c(xmin, ymin)
    )))
  )
}

# If just the points are provided, guess the boundary
if (is.null(boundary) && is.null(window)) {

  xy <- as.matrix(xy)
  stopifnot(ncol(xy) == 2L, nrow(xy) >= 2L)
  storage.mode(xy) <- "double"

  knn <- FNN::get.knn(xy, k = 1)
  buffer <- q * median(knn$nn.dist[, 1], na.rm = TRUE)
  delta <- buffer / 10

  cell_x <- floor(xy[, 1] / delta)
  cell_y <- floor(xy[, 2] / delta)
  cell_id <- paste(cell_x, cell_y, sep = "_")
  xy <- xy[!duplicated(cell_id), , drop = FALSE]

  pts <- sf::st_as_sf(
    as.data.frame(xy),
    coords = 1:2
  )

  boundary <- sf::st_buffer(pts, dist = buffer + delta)
  boundary <- sf::st_union(boundary)
  boundary <- sf::st_simplify(boundary, dTolerance = delta, preserveTopology = TRUE)
}

```

```

}

if (is.null(boundary)) {
  stop("Provide one of: boundary, window, or xy.")
}

# Final polygon cleanup
boundary <- sf::st_geometry(boundary)
boundary <- sf::st_make_valid(boundary)

gtype <- sf::st_geometry_type(boundary)
if (any(gtype == "GEOMETRYCOLLECTION"))
  boundary <- sf::st_collection_extract(boundary, "POLYGON")

if (length(boundary) == 0 || all(sf::st_is_empty(boundary)))
  stop("Boundary does not contain polygon geometry.")

boundary <- sf::st_union(boundary)
boundary <- sf::st_make_valid(boundary)
boundary
}

```

Now, a useful auxiliary function to split a given set of points into “out”, “edge” and “core”, given a boundary and a distance threshold. It takes advantage of the fast calculation of distances (function above). This function is used to concentrate efforts of numerical estimates for the points near the boundaries, using the well-known formula for the circle area for points far from the boundary.

```

# Clasify points based on a given boundary.
# The boundary can be provided as a sfc_POLYGON of arbitrary shape
# or a rectangular window (xmin, ymin, xmax, ymax). If it is not
# provided it will be assumed rectangular around the points provided.

core_vs_edge <- function(xy,
                          boundary = NULL,
                          window = NULL,
                          radius) {

  # Parameters:
  # xy: a matrix or data.frame with x and y coordinates in 1st and 2nd column respectively
  # boundary: boundary defined as a sfc_POLYGON object, for boundaries of any shape
  # window: alternative to "boundary" for rectangular boundaries, as c(xmin, ymin, xmax, ymax)

```

```

# radius: the distance to the boundary that separates core from edge points
# assume_inside: will accelerate if we know all xy points are inside the boundary

# Confirm 2-columns matrix for the point coordinates
xy <- as.matrix(xy)
stopifnot(ncol(xy) == 2L)
storage.mode(xy) <- "double"

# Confirm or guess boundary:
boundary <- get_boundary(boundary = boundary, window = window, xy = xy)

# Determine which points are inside:
pts <- sf::st_as_sf(
  as.data.frame(xy),
  coords = 1:2,
  crs = sf::st_crs(boundary),
  remove = FALSE
)
inside <- lengths(sf::st_intersects(pts, boundary)) > 0

# Densify boundary once, then use fast_distances_up_to_r()
boundary <- sf::st_boundary(boundary)
boundary <- sf::st_segmentize(boundary, dfMaxLength = radius / 10)

boundary_xy <- sf::st_coordinates(boundary)[, c("X", "Y"), drop = FALSE]
boundary_xy <- unique(boundary_xy)
storage.mode(boundary_xy) <- "double"

# Identify points near the boundary:
near <- rep(FALSE, nrow(xy))
ii <- which(inside)
if (length(ii)) {
  d <- fast_distances_up_to_r(
    query = xy[ii, , drop = FALSE],
    reference = boundary_xy,
    radius = 1.05 * radius # small safety margin
  )
  near[ii[unique(d$query_i)]] <- TRUE
}

# Build output into three classes:

```

```

location <- rep(NA, length(inside))
location[!inside] <- "out"
location[inside & near] <- "edge"
location[inside & !near] <- "core"
location
}

```

The neighborhood analyses are scale-explicit, looking at circles or rings of increasing sizes. The user may not have a predefined set of scales at which to measure the metrics of interest. This function implements useful defaults, given a set of query points:

```

get_radial <- function(query, rmax = NA, m = 512, case = c("circles", "rings"))
{
  # Default case: circles
  if(length(case) != 1) case = "circles"

  # Implements same defaults as Ripley's K function in spatstat
  if(is.na(rmax))
  {
    a = diff(range(query[,1], na.rm = TRUE))
    b = diff(range(query[,2], na.rm = TRUE))
    lambda = nrow(query) / (a * b)
    rmax = min(c(a/4, b/4, sqrt(1000 / (pi * lambda))))
  }

  # Default for circles: simple equal increments in radii
  radii.c <- seq(from = rmax/m, to = rmax, by = rmax/m)

  # Default for rings: rings of equal area
  # (It returns the outer radii: the inner radii is 0 and the same sequence but shifted)
  radii.r <- rmax*sqrt((1:m)/m)

  # Output
  if(case == "circles") radii <- radii.c
  if(case == "rings")   radii <- radii.r
  radii
}

```

The fundamental piece of information to control for edge effects are the areas of the neighborhoods at different scales, both for the “core” points (easy) and the “edge” points (difficult). The code can handle irregular shapes, estimating areas numerically within a tolerance to error, defined by the user. To do so, it creates dense grids (but not excessively dense) within the

valid boundary, and then counts points in the grid within the provided radii. The function below integrates useful defaults and rules of thumb.

```
get_area_of_neighborhoods <- function(query = NULL,
                                       boundary = NULL,
                                       window = NULL,
                                       radii = NULL,
                                       rmax = NA,
                                       m = 512,
                                       case = c("circles", "rings"),
                                       tol = 0.05)
{
  # query: The set of focal or query points.
  #       2-columns table with x and y coordinates in 1st and 2nd column, respectively.
  # boundary: boundary defined as a sfc_POLYGON object, for boundaries of any shape
  # window: rectangular, alternative to "boundary", as c(xmin, ymin, xmax, ymax)
  # radii: defines the scales of the circular neighborhoods.
  # rmax: maximum radius for neighborhoods, if radii not provided
  # m: number of scales to evaluate, if radii not provided
  # case: circular vs. ring-shaped neighborhoods: will affect area and default radii
  # tol: the accepted error in area, 5% by default.

  # Default case: circles
  if(length(case) != 1) case = "circles"

  # Confirm or guess boundary
  boundary <- get_boundary(boundary = boundary, window = window, xy = query)
  bb <- sf::st_bbox(boundary)
  xmin = bb["xmin"]
  xmax = bb["xmax"]
  ymin = bb["ymin"]
  ymax = bb["ymax"]

  # Define radii, if the user has not provided them yet:
  if(is.null(radii))
    radii <- get_radii(query = query, rmax = rmax, m = m, case = case)
  m = length(radii)
  rmax = max(radii)

  # The areas of neighborhoods at different scales will
  # repeat similar calculations, and will use different
  # grids for each scale. The alternative is to commit
```

```

# to a very dense grid with low error in the worst case
# scenario, but costly calculations when it is not really
# needed to achieve a given tolerance.

# This runs a loop across scales. Before the loop,
# it creates empty objects to store results.
areas <- matrix(NA, nrow = nrow(query), ncol = m) # point x scale matrix
steps <- rep(NA, m)
for(i in 1:m)
{
  # The target scale
  radius = radii[i]

  # Define the resolution of the grid.
  # Our approach is the Gauss circle problem,
  # and the associated error is not perfectly
  # known, but relatively easy to estimate numerically.
  # There are useful rules of thumb, empirical results, etc.

  # This is a useful heuristic for the step associated
  # with a given error in the case "points within circles",
  # when the points come from a regular grid:
  if(case == "circles")
    step = radius * (1.772 * tol)^(2/3)

  # The "points within rings" follows a similar
  # logic, but adds more penalization, because
  # of the greater amount of perimeter.
  # Note that outer rings, if of equal area,
  # are thinner and thinner, and perimeter:area
  # ratio grows fast, thus demanding denser and
  # denser grids to constrain error within tolerance.
  if(case == "rings")
  {
    if(i == 1) r0 = 0
    if(i >= 2) r0 = radii[i-1]
    r1 = radius
    r_area = sqrt(r1^2 - r0^2) # area-equivalent circle radius
    B_ring = (r1 + r0) / r_area # boundary inflation vs. equal-area circle
    step = r_area * (1.772 * tol)^(2/3) / B_ring^(1/3)
  }
}

```

```

# Because the regular grid is the "easy" case,
# we need to improve the resolution so it
# returns the same error around the (possibly complex) boundary.
# This is a conservative heuristic derived from Steiner / parallel-body argument:
K = 1 + 4 / pi
step = step / K
steps[i] <- step

# Put all the areas together: initialize with full circles,
# then overwrite with the areas calculated for the edge cases:
A0 = pi*radius^2
A <- rep(A0, nrow(query))
ceo_query <- core_vs_edge(xy = query, boundary = boundary, radius = radius)
query_edge <- query[ceo_query == "edge",,drop = FALSE]

if(nrow(query_edge) > 0)
{
  # Note the 2r distance needed for the grid to cover the entire
  # neighborhood of query points that are at r distance from the edge.
  grid <- expand.grid(x = seq(from = xmin - step/2, to = xmax + step/2, by = step),
                    y = seq(from = ymin - step/2, to = ymax + step/2, by = step))

  ceo_grid <- core_vs_edge(xy = grid, boundary = boundary, radius = 2*radius)
  grid_edge <- grid[ceo_grid == "edge",,drop = FALSE]

  # Use the neighbor-identifying workhorse to identify points in the grid that
  # are inside the neighborhoods. Their number is a proxy for neighborhood area.
  d <- fast_distances_up_to_r(query = query_edge,
                             reference = grid_edge,
                             radius = radius)

  A[ceo_query == "edge"] <- pmin(A0,
                                tabulate(d$query_i, nbins = nrow(query_edge)) * step^2)
}

areas[,i] <- A
print(paste0(i, "/", m))
}

# The ring case is solved from the circles case,
# just by sequential subtraction:
if(case == "rings")

```

```

{
  large <- areas[, 2:m, drop = FALSE]
  small <- areas[, 1:(m-1), drop = FALSE]
  areas <- cbind(areas[,1], large - small) # the inner ring is a circle, zero subtraction
}

# Output:
if(case == "circles")
  scales <- data.frame(radius = radii,
                      grid_step = steps,
                      grid_points_per_circle = pi * radii^2 / steps^2)

if(case == "rings")
{
  inner_radii <- c(0, radii[-m])
  scales <- data.frame(inner_radius = inner_radii,
                      outer_radius = radii,
                      grid_step = steps,
                      grid_points_per_ring = pi * (radii^2 - inner_radii^2) / steps^2)
}

list(case = case, scales = scales, areas = areas)
}

```

4 Examples

This section uses real data to show how to perform some spatial analyses. It includes:

- Example 1: Ripley's K and Besag's L, using code from outside
- Example 2: Aggregation within one species, using number of individuals or accumulated basal area
- Example 3: Quantity of conspecifics and heterospecifics
- Example 4: Associations between pairs of species
- Example 5: Diversity in the neighborhood of different species

Note that, except for the first example, the filtering, categorizations, summary metrics, and null expectations, are all outside the functions themselves. It is the responsibility of the user to define these things.

```

# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())

```

```
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Load data from the Luquillo forest plot, public domain, as an example
load(paste0(path, "data_examples/luquillo.full2.rdata"))
data <- luquillo.full2
dim(data)
```

```
[1] 125446      19
```

```
head(data)
```

	treeID	stemID	tag	StemTag	sp	quadrat	gx	gy	DBHID	CensusID	dbh	pom
1	1	1	-110496	-110496	CECSCH	622	NA	NA	103376		2	NA <NA>
3	2	2	-11392	-11392	PSYBER	503	NA	NA	103377		2	NA <NA>
5	3	3	-114036	-114036	PSYBER	1223	NA	NA	103378		2	NA <NA>
7	4	4	-11756	-11756	CECSCH	901	NA	NA	103379		2	NA <NA>
9	5	5	-12584	-12584	CECSCH	801	NA	NA	103380		2	NA <NA>
11	6	6	-15558	-15558	UNIDEN	303	NA	NA	103381		2	NA <NA>

	hom	ExactDate	DFstatus	codes	nostems	status	date
1	NA	1996-07-03	dead	MAIN;DEADT	1	D	13333
3	NA	1995-02-09	dead	MAIN;DEADT	1	D	12823
5	NA	1996-08-02	dead	MAIN;DEADT	1	D	13363
7	NA	1995-02-23	dead	MAIN;DEADT	1	D	12837
9	NA	1995-02-13	dead	MAIN;DEADT	1	D	12827
11	NA	1995-02-03	dead	MAIN;DEADT	1	D	12817

```
colnames(data)
```

```
[1] "treeID"      "stemID"      "tag"         "StemTag"     "sp"          "quadrat"
[7] "gx"         "gy"         "DBHID"      "CensusID"    "dbh"         "pom"
[13] "hom"        "ExactDate"  "DFstatus"   "codes"       "nostems"     "status"
[19] "date"
```

```
summary(data)
```

	treeID		stemID		tag		StemTag
Min.	:	1	Min.	:	1	Min.	:-172721
1st Qu.:	31363		1st Qu.:	29528		1st Qu.:	41423
Median	:	62726	Median	:	57530	Median	:
							84202
							66158

Mean : 62726	Mean : 57731	Mean : 84106	Mean : 64320
3rd Qu.: 94089	3rd Qu.: 86554	3rd Qu.: 124460	3rd Qu.: 97486
Max. :125450	Max. :114032	Max. : 175926	Max. : 146191
	NA's :30956		NA's :30956

sp	quadrat	gx	gy
Length:125446	Min. : 101	Min. : 0.01	Min. : 0.0
Class :character	1st Qu.: 522	1st Qu.: 86.43	1st Qu.:127.7
Mode :character	Median : 922	Median :164.29	Median :265.2
	Mean : 920	Mean :163.49	Mean :254.9
	3rd Qu.:1316	3rd Qu.:243.75	3rd Qu.:380.8
	Max. :1625	Max. :319.98	Max. :499.9
	NA's :30956	NA's :83870	NA's :83870

DBHID	CensusID	dbh	pom
Min. :103376	Min. :2	Min. : 5.40	Length:125446
1st Qu.:132902	1st Qu.:2	1st Qu.: 15.70	Class :character
Median :160900	Median :2	Median : 27.50	Mode :character
Mean :161103	Mean :2	Mean : 63.61	
3rd Qu.:189924	3rd Qu.:2	3rd Qu.: 79.00	
Max. :217400	Max. :2	Max. :1515.00	
NA's :30956	NA's :30956	NA's :58283	

hom	ExactDate	DFstatus	codes
Min. :0.010	Length:125446	Length:125446	Length:125446
1st Qu.:1.300	Class :character	Class :character	Class :character
Median :1.300	Mode :character	Mode :character	Mode :character
Mean :1.305			
3rd Qu.:1.300			
Max. :3.900			
NA's :58288			

nostems	status	date
Min. : 1.000	Length:125446	Min. :12717
1st Qu.: 1.000	Class :array	1st Qu.:12984
Median : 1.000	Mode :character	Median :13205
Mean : 1.207		Mean :13152
3rd Qu.: 1.000		3rd Qu.:13310
Max. :29.000		Max. :14013
NA's :30956		NA's :30956

```
# Spatial analyses almost invariably refer to individuals, not stems,
# so we should go from stem-level information to individual-level information.

# In real applications, you should pay attention and do proper data aggregation
# from the stem level to the individual level. That is more difficult than
```

```
# it seems. I do not want to over-complicate the example here, so I simply
# subset the dataset for the main stems of each individual, taking
# advantage of the codes. This is something that was important in the old
# workflows but NOT a general recommendation.
```

```
main <- grep("MAIN", data$codes)
data <- data[main,]
dim(data)
```

```
[1] 92456    19
```

4.1 Example 1: Ripley's K and Besag's L

Ripley's K function measures how many neighboring points occur within distance r , relative to what is expected under complete spatial randomness. Values above expectation indicate clustering; values below expectation indicate regularity or inhibition. Besag's L is a variance-stabilized transformation of K, often plotted as $L(r) - r$, making deviations from randomness easier to interpret.

```
# We use one species as a example
sp = "PREMON" # Prestoea, the most common palm

# The gold standard is in spatstat packages:
library(spatstat.geom)
```

```
Warning: package 'spatstat.geom' was built under R version 4.5.2
```

```
Loading required package: spatstat.data
```

```
Loading required package: spatstat.univar
```

```
Warning: package 'spatstat.univar' was built under R version 4.5.2
```

```
spatstat.univar 3.1-7
```

```
spatstat.geom 3.7-3
```

```
library(spatstat.explore)
```

Warning: package 'spatstat.explore' was built under R version 4.5.2

Loading required package: spatstat.random

Warning: package 'spatstat.random' was built under R version 4.5.2

spatstat.random 3.4-5

Loading required package: nlme

Warning: package 'nlme' was built under R version 4.5.2

spatstat.explore 3.8-0

```
# This is the rectangular observation window:
```

```
W <- owin(  
  xrange = round(range(data$gx, na.rm = TRUE)),  
  yrange = round(range(data$gy, na.rm = TRUE))  
)
```

```
# Point pattern spatial object for both species;  
# we focus only on the living trees.
```

```
x <- data$gx[data$sp == sp & data$status == "A"]
```

```
y <- data$gy[data$sp == sp & data$status == "A"]
```

```
PPP <- ppp(x = x, y = y, window = W)
```

Warning in ppp(x = x, y = y, window = W): 56 out of 6014 points had NA or NaN coordinate values, and were discarded

```
# Aggregation at different scales:
```

```
radii <- seq(from = 0, to = 50, by = 0.5)
```

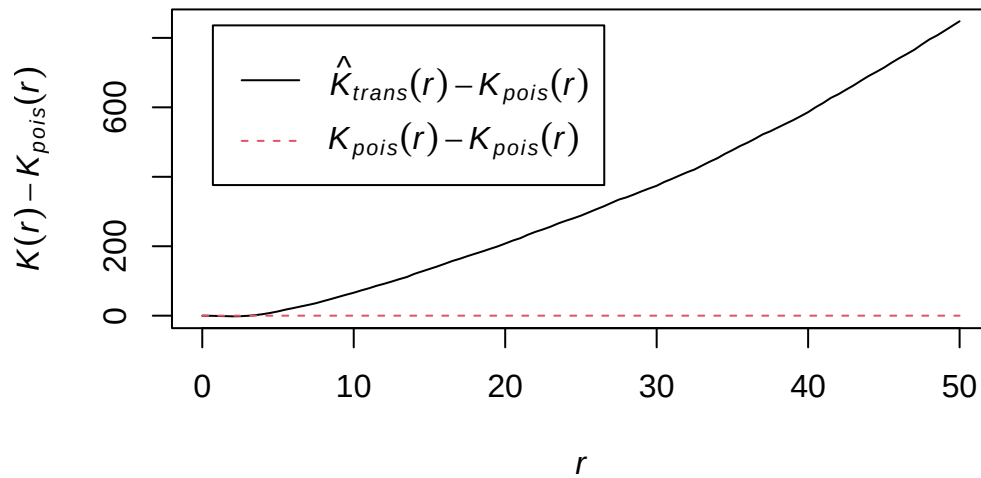
```
K <- Kest(PPP, r = radii, correction = "translate")
```

```
L <- Lest(PPP, r = radii, correction = "translate")
```

```
# Results:
```

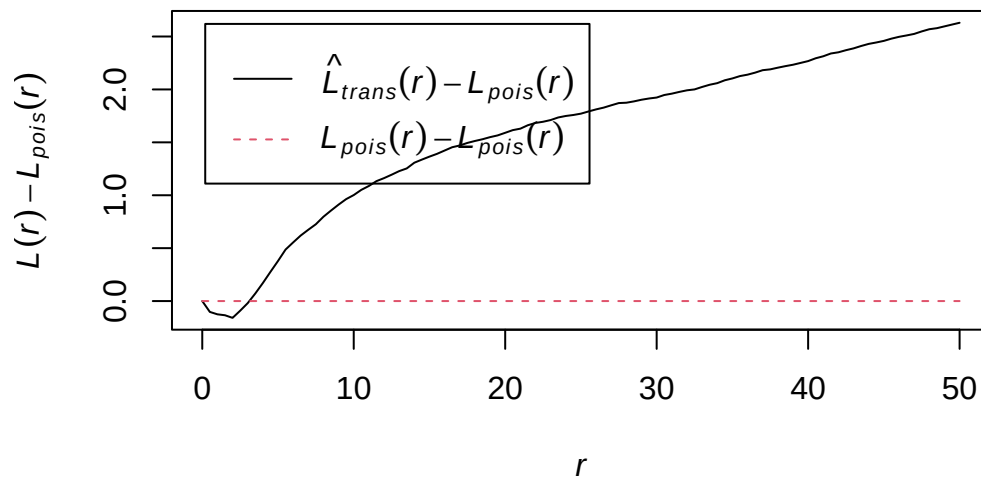
```
plot(K, . - theo ~ r, main = "Ripley's K: K(r) - Kpois(r)")
```

Ripley's K: $K(r) - K_{pois}(r)$



```
plot(L, . - theo ~ r, main = "Besag's L: L(r) - r")
```

Besag's L: $L(r) - r$



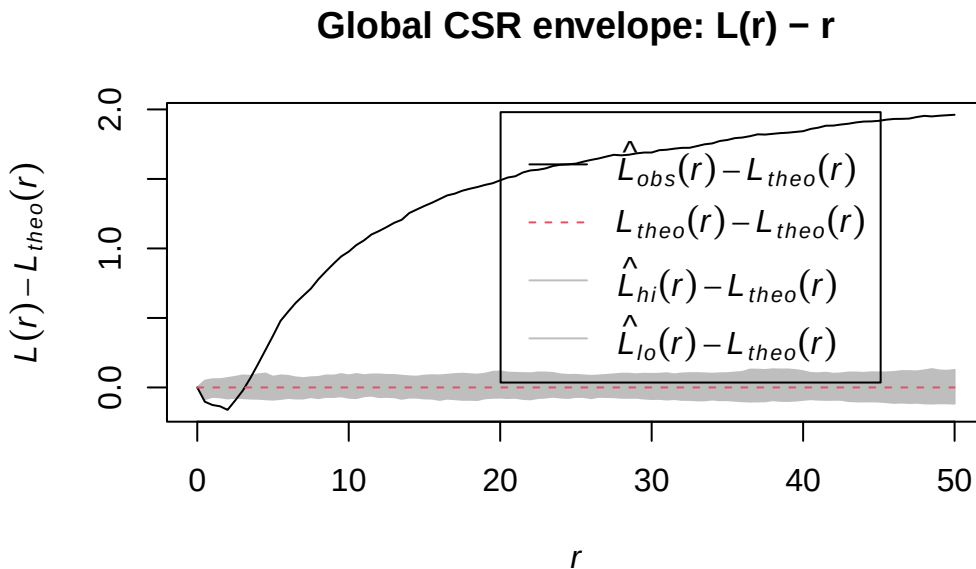
```
# With simulation envelopes, only for Besag's L,
# which is easier to interpret visually:
L.env <- envelope(PPP, fun = Lest, r = radii, nsim = 199)
```

```
Generating 199 simulated realisations of CSR ...
1, 2, 3, 4.6.8.10.12.14.16.18.20.22.24.26.28.30.32.34
```

```
.36.38.40.42.44.46.48.50.52.54.56.58.60.62.64.66.68.70.72.74
.76.78.80.82.84.86.88.90.92.94.96.98.100.102.104.106.108.110.112.114
.116.118.120.122.124.126.128.130.132.134.136.138.140.142.144.146.148.150.152.154
.156.158.160.162.164.166.168.170.172.174.176.178.180.182.184.186.188.190.192.194
.196.198
199.
```

Done.

```
plot(L.env, . - theo ~ r, main = "Global CSR envelope: L(r) - r")
```



The analysis above shows some “repulsion” at short distances for the individuals of *Prestoea*. That is: fewer individuals at distances 0-3 m or so than expected by chance. This is not very common... In general tree species tend to be aggregated at most scales. This makes me wonder: What are the species that show “avoidance” (fewer individuals than expected) and at what scales? This is an example on how to get that:

```
# What species, and at what scales, have fewer neighbors than expected?
# We exclude species with too few individuals:
min.n = 100
a <- sort(table(data$sp[data$status == "A"]), decreasing = TRUE)
spp <- names(which(a >= min.n))

# This is the rectangular observation window:
W <- owin(
  xrange = round(range(data$gx, na.rm = TRUE)),
```

```

    yrange = round(range(data$gy, na.rm = TRUE))
  )

# Scales of observation, centered over short distances:
radii <- seq(from = 0, to = 20, by = 0.2)

# Run loop through species:
segregations <- NULL
for(sp in spp)
{
  x.sp <- data$gx[data$sp == sp & data$status == "A"]
  y.sp <- data$gy[data$sp == sp & data$status == "A"]
  PPP.sp <- ppp(x = x.sp, y = y.sp, window = W)
  L.sp.env <- envelope(PPP.sp, fun = Lest, r = radii, nsim = 49)

  w <- which(L.sp.env$obs < L.sp.env$lo)
  if(length(w) > 0)
  {
    add <- data.frame(sp = sp, r = L.sp.env$r[w])
    segregations <- rbind(segregations, add)
  }
  print(paste0(which(spp == sp), "/", length(spp)))
}

```

Warning in ppp(x = x.sp, y = y.sp, window = W): 10961 out of 12394 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
 41, 42, 43, 44, 45, 46, 47, 48,
 49.

Done.

[1] "1/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 5062 out of 6483 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,

21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "2/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 56 out of 6014 points had NA or
NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "3/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 2495 out of 4993 points had NA
or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "4/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 386 out of 3347 points had NA
or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "5/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 234 out of 3190 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "6/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 1391 out of 2727 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "7/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 496 out of 2180 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "8/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 2013 out of 2126 points had NA or NaN coordinate values, and were discarded

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "9/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 282 out of 1685 points had NA
or NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "10/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 49 out of 1641 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "11/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 68 out of 1631 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

Done.

[1] "12/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 90 out of 1270 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "13/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 305 out of 1179 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "14/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 198 out of 1135 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "15/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 115 out of 818 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "16/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 58 out of 792 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "17/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 152 out of 772 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "18/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 338 out of 747 points had NA or NaN coordinate values, and were discarded

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "19/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 84 out of 745 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "20/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 68 out of 635 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "21/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 43 out of 581 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

Done.

[1] "22/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 50 out of 489 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "23/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 116 out of 475 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "24/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 56 out of 405 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "25/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 67 out of 390 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "26/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 60 out of 350 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "27/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 174 out of 331 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "28/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 68 out of 305 points had NA or NaN coordinate values, and were discarded

```
Generating 49 simulated realisations of CSR ...  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,  
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,  
41, 42, 43, 44, 45, 46, 47, 48,  
49.
```

```
Done.  
[1] "29/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 11 out of 295 points had NA or  
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,  
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,  
41, 42, 43, 44, 45, 46, 47, 48,  
49.
```

```
Done.  
[1] "30/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 132 out of 292 points had NA or  
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,  
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,  
41, 42, 43, 44, 45, 46, 47, 48,  
49.
```

```
Done.  
[1] "31/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 37 out of 282 points had NA or  
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,  
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,  
41, 42, 43, 44, 45, 46, 47, 48,  
49.
```

Done.

[1] "32/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 20 out of 261 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "33/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 83 out of 244 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "34/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 68 out of 237 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "35/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 31 out of 235 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "36/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 50 out of 233 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "37/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 44 out of 232 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "38/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 215 out of 222 points had NA or NaN coordinate values, and were discarded

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "39/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 43 out of 205 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "40/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 37 out of 200 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "41/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 4 out of 189 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

Done.

[1] "42/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 12 out of 186 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "43/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 16 out of 186 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "44/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 47 out of 182 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "45/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 54 out of 176 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "46/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 10 out of 172 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "47/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 25 out of 155 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "48/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 3 out of 153 points had NA or NaN coordinate values, and were discarded

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "49/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 13 out of 141 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "50/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 25 out of 137 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
[1] "51/56"
```

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 10 out of 131 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

Done.

[1] "52/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 10 out of 119 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "53/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 27 out of 116 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "54/56"

Warning in ppp(x = x.sp, y = y.sp, window = W): 108 out of 111 points had NA or NaN coordinate values, and were discarded

Generating 49 simulated realisations of CSR ...

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.

Done.

[1] "55/56"

```
Warning in ppp(x = x.sp, y = y.sp, window = W): 13 out of 101 points had NA or
NaN coordinate values, and were discarded
```

```
Generating 49 simulated realisations of CSR ...
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48,
49.
```

```
Done.
```

```
[1] "56/56"
```

```
# How frequent is it?
```

```
nrow(segregations) # number of scales with significant segregations
```

```
[1] 13
```

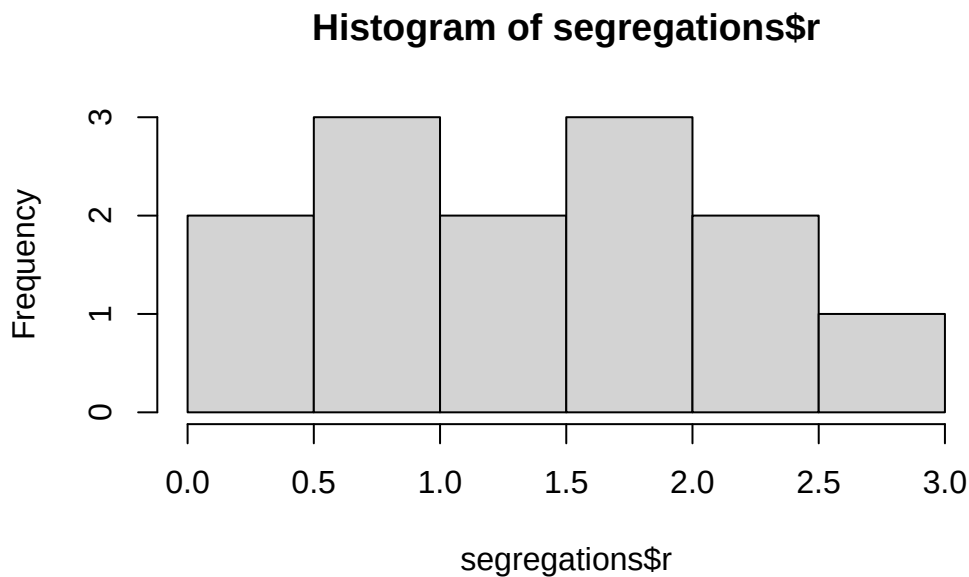
```
length(radii) * length(spp) # number of scales evaluated
```

```
[1] 5656
```

```
segregations
```

	sp	r
1	PREMON	0.2
2	PREMON	0.4
3	PREMON	0.6
4	PREMON	0.8
5	PREMON	1.0
6	PREMON	1.2
7	PREMON	1.4
8	PREMON	1.6
9	PREMON	1.8
10	PREMON	2.0
11	PREMON	2.2
12	PREMON	2.4
13	PREMON	2.6

```
# What are the species and scales involved?  
hist(segregations$r)
```



```
table(segregations$sp)
```

```
PREMON  
13
```

```
# Etc.
```

4.2 Example 2: Aggregation within species

This example uses the new code to do something similar, but highlighting the flexibility in the definition of boundary (complex) and the abundance/summary metric (outside the functions themselves). It also showcases the default behaviour of some of the functions.

```
library(sf)
```

```
Warning: package 'sf' was built under R version 4.5.2
```

```
Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```

# Study aggregation of living Prestoea montana, but only within a low-disturbance portion of

low_disturbance_xy <- rbind(
  c(0, 0),
  c(320, 0),
  c(320, 100),
  c(300, 100),
  c(300, 120),
  c(160, 120),
  c(160, 140),
  c(140, 140),
  c(140, 160),
  c(120, 160),
  c(120, 180),
  c(100, 180),
  c(100, 200),
  c(80, 200),
  c(80, 220),
  c(60, 220),
  c(60, 240),
  c(40, 240),
  c(40, 260),
  c(0, 260),
  c(0, 0)
)

boundary <- sf::st_sfc(st_polygon(list(low_disturbance_xy)))

# In general it will be useful to get subsets of the dataset with all the variables,
# so it is easier to align from a given point to a given value in the table.
sp = "PREMON"
keep1 <- data$sp == sp
keep2 <- data$status == "A"
keep3 <- !is.na(data$gx)
keep4 <- !is.na(data$gy)
keep <- keep1 & keep2 & keep3 & keep4
query_data <- data[keep,]

pts <- sf::st_as_sf(query_data[,c("gx", "gy")], coords = 1:2)
inside <- lengths(sf::st_intersects(pts, boundary)) > 0

query_data <- query_data[inside,]

```

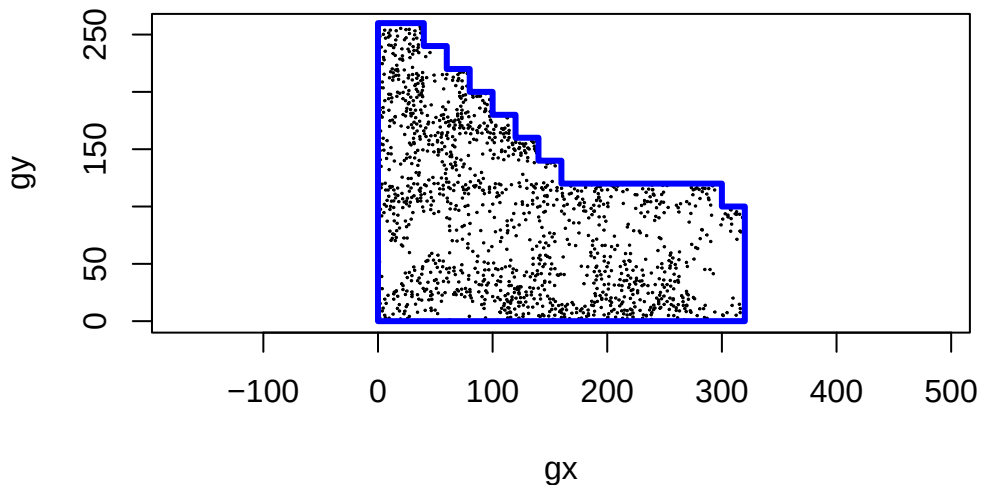
```

query <- query_data[,c("gx", "gy")] # do not break alignment down the road!

# (In general, we would do the same for a reference subset, but here we are
# exploring conspecific aggregation, so query = reference).

plot(query, asp = 1, cex = 0.1)
plot(boundary, asp = 1, border = "blue", lwd = 3, add = TRUE)

```



```

# Scales to study?
radii <- get_radii(query = query, m = 50, case = "circles")
max(radii)

```

```
[1] 64.3575
```

```

# Calculate the areas of the neighborhoods:
A <- get_area_of_neighborhoods(query = query,
                                boundary = boundary,
                                radii = radii,
                                case = "circles",
                                tol = 0.05)

```

```

[1] "1/50"
[1] "2/50"
[1] "3/50"
[1] "4/50"
[1] "5/50"

```

[1] "6/50"
[1] "7/50"
[1] "8/50"
[1] "9/50"
[1] "10/50"
[1] "11/50"
[1] "12/50"
[1] "13/50"
[1] "14/50"
[1] "15/50"
[1] "16/50"
[1] "17/50"
[1] "18/50"
[1] "19/50"
[1] "20/50"
[1] "21/50"
[1] "22/50"
[1] "23/50"
[1] "24/50"
[1] "25/50"
[1] "26/50"
[1] "27/50"
[1] "28/50"
[1] "29/50"
[1] "30/50"
[1] "31/50"
[1] "32/50"
[1] "33/50"
[1] "34/50"
[1] "35/50"
[1] "36/50"
[1] "37/50"
[1] "38/50"
[1] "39/50"
[1] "40/50"
[1] "41/50"
[1] "42/50"
[1] "43/50"
[1] "44/50"
[1] "45/50"
[1] "46/50"
[1] "47/50"
[1] "48/50"

```
[1] "49/50"
[1] "50/50"
```

```
# Identify the neighbors:
# No need to loop through scales now, unless there are memory limitations.
# We do it once, for the maximum radius. We subset and process later.
# Because we want aggregation of one species with itself, then reference = query.
neighbors <- fast_distances_up_to_r(query = query, reference = query, radius = max(radii))
dim(neighbors)
```

```
[1] 374331      3
```

```
head(neighbors)
```

	query_i	reference_j	dist
	<int>	<int>	<num>
1:	1	3	4.960262
2:	1	1398	4.780387
3:	1	2	3.769695
4:	1	811	3.491848
5:	1	810	3.861204
6:	1	1	0.000000

```
# Note: because reference = query in this application, every individual
# appears as a neighbor of itself. We can remove them:
keep <- which(neighbors$query_i != neighbors$reference_j)
neighbors <- neighbors[keep,]

# Then, we can summarize those neighborhoods any way we want.
# The data.table syntax is faster and, if you can, do it that way.
# There are different tools for data aggregation in R, and
# you may have to explore them, depending on your goals, size of dataset, etc.
# Here I use simple base code as a template, going scale by scale.

# First, I externalize the calculation of basal area
# (this would be done on "reference" in general, but
# for conspecifics reference = query)
query_data$BA <- pi*(query_data$dbh/2)^2
neighbors$BA_j <- query_data$BA[neighbors$reference_j]
head(neighbors)
```

	query_i	reference_j	dist	BA_j
	<int>	<int>	<num>	<num>
1:	1	3	4.960262	22698.007
2:	1	1398	4.780387	12076.282
3:	1	2	3.769695	20106.193
4:	1	811	3.491848	17203.361
5:	1	810	3.861204	8171.282
6:	1	809	5.868219	10386.891

```
# Then, I create empty objects to store results:
N.total.in.neighborhoods <- matrix(NA, nrow = nrow(query), ncol = length(radii))
BA.total.in.neighborhoods <- matrix(NA, nrow = nrow(query), ncol = length(radii))

# Then, I look through scales:
# (I use "k" to avoid confusion with {i,j} pairs of neighbors)
for(k in 1:length(radii))
{
  radius = radii[k]
  sub <- neighbors[neighbors$dist <= radius,]

  # Counting: how many times "i" appears = number of j neighbors
  counts <- tabulate(sub$query_i, nbins = nrow(query))
  N.total.in.neighborhoods[,k] <- counts

  # Sum basal areas:
  agg.ba <- aggregate(x = sub$BA_j, by = list(sub$query_i), FUN = "sum", na.rm = TRUE)
  BA.total.in.neighborhoods[agg.ba[,1], k] <- agg.ba[,2]

  # Track progress:
  print(paste0(k, "/", length(radii)))
}
```

```
[1] "1/50"
[1] "2/50"
[1] "3/50"
[1] "4/50"
[1] "5/50"
[1] "6/50"
[1] "7/50"
[1] "8/50"
[1] "9/50"
[1] "10/50"
```

[1] "11/50"
[1] "12/50"
[1] "13/50"
[1] "14/50"
[1] "15/50"
[1] "16/50"
[1] "17/50"
[1] "18/50"
[1] "19/50"
[1] "20/50"
[1] "21/50"
[1] "22/50"
[1] "23/50"
[1] "24/50"
[1] "25/50"
[1] "26/50"
[1] "27/50"
[1] "28/50"
[1] "29/50"
[1] "30/50"
[1] "31/50"
[1] "32/50"
[1] "33/50"
[1] "34/50"
[1] "35/50"
[1] "36/50"
[1] "37/50"
[1] "38/50"
[1] "39/50"
[1] "40/50"
[1] "41/50"
[1] "42/50"
[1] "43/50"
[1] "44/50"
[1] "45/50"
[1] "46/50"
[1] "47/50"
[1] "48/50"
[1] "49/50"
[1] "50/50"

```
N.total.in.neighborhoods[1:10, 1:10]
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
[1,]	0	0	3	5	6	7	7	7	9	13
[2,]	0	0	1	3	3	7	7	8	8	9
[3,]	0	1	3	4	5	9	13	13	15	16
[4,]	0	1	1	2	4	8	10	10	15	17
[5,]	0	0	0	2	3	5	8	9	13	16
[6,]	0	0	0	2	2	4	5	7	11	12
[7,]	0	0	1	3	3	4	8	9	10	14
[8,]	0	0	2	2	3	5	6	11	14	16
[9,]	0	0	1	4	6	8	9	11	14	19
[10,]	0	0	1	2	4	9	12	15	19	21

```
BA.total.in.neighborhoods[1:10, 1:10]
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]
[1,]	NA	NA	45480.84	80255.13	90642.02	109511.21	109511.2	109511.2
[2,]	NA	NA	25446.90	56392.37	56392.37	114851.92	114851.9	131823.6
[3,]	NA	10386.89	30634.46	56081.36	74950.55	145022.20	190144.1	190144.1
[4,]	NA	10751.32	10751.32	27037.33	53540.59	119170.03	144950.7	144950.7
[5,]	NA	NA	NA	27954.68	46823.87	69287.04	131324.9	151431.0
[6,]	NA	NA	NA	42418.57	42418.57	80929.00	108100.6	136821.9
[7,]	NA	NA	27759.11	72134.11	72134.11	81986.14	149454.2	160205.5
[8,]	NA	NA	52618.54	52618.54	76397.25	111986.00	121838.0	195722.0
[9,]	NA	NA	27759.11	81443.43	122658.77	146827.83	164031.2	191001.8
[10,]	NA	NA	15836.77	34222.15	74261.75	141769.08	185071.2	244065.6

	[,9]	[,10]
[1,]	149247.6	201614.9
[2,]	131823.6	142574.9
[3,]	227297.4	247153.0
[4,]	229542.0	271846.7
[5,]	205545.8	264811.9
[6,]	181510.2	205288.9
[7,]	177642.1	239650.1
[8,]	243537.8	267157.1
[9,]	234031.4	301621.2
[10,]	315783.5	355292.9

Having the available area for each neighborhood, and then expressing metrics on a per area basis, can be one explicit way of accounting for edge effects. This was the approach taken in old CTFS code.

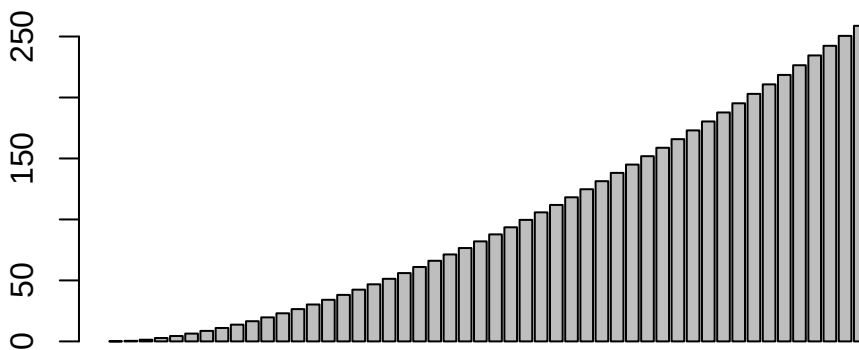
Alternatively, one can control for edge effects using numerical null models, and avoiding the calculation of areas entirely. The null expectations already incorporate the fact that points near the boundaries will tend to have fewer neighbors (and thus lower basal area, fewer species, etc). If deviations are standardized, they will solve for edge effects.

I present a template on how to do that, for the simple case of number of neighbors. However, this would be most useful for metrics that do *not* grow linearly with area, like species diversity, and that cannot be directly corrected by expressing in a per-area basis.

```
# Null models compare observations (one instance) with expectations (many instances).
# If reality does not deviate much from the expectations by the null model, then
# the researcher cannot reject the null hypothesis modelled by the null model.
# If reality deviates from the expectations, then we can reject the null hypothesis.

# Let's say we are interested in the mean number of conspecific
# neighbors at different scales, and whether that is more or
# less than the expected by chance.
obs <- colMeans(N.total.in.neighborhoods) # observations

barplot(obs)
```



```
# The expectations will be stores in a matrix, with one row per simulation:
nsim = 9 # 999 or so for real applications -- small numbers for examples and rendering
null <- matrix(NA, ncol = length(radii), nrow = nsim)

# Get the null expectations
for(sim in 1:nsim)
{
  # The simplest of the expectations is uniform random distribution of points,
  # within the valid boundaries. Easy to get from sf::st_sample.
  null_xy <- sf::st_sample(x = get_boundary(boundary),
                           size = nrow(query),
```

```

        type = "random",
        exact = TRUE)
null_xy <- sf::st_coordinates(null_xy)

# Expected conspecific neighbors, if the species was randomly distributed:
null_neighbors <- fast_distances_up_to_r(query = null_xy,
                                         reference = null_xy,
                                         radius = max(radii))

keep <- which(null_neighbors$query_i != null_neighbors$reference_j)
null_neighbors <- null_neighbors[keep,]

# Then same loop through scales as before:
for(k in length(radii):1)
{
  radius = radii[k]
  null_sub <- null_neighbors[null_neighbors$dist <= radius,]
  null_counts <- tabulate(null_sub$query_i, nbins = nrow(null_xy))
  null[sim,k] <- mean(null_counts) # store in the right place
}
print(sim)
}

```

```

[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9

```

```

# There are different ways of comparing observations with expectations.
# A common one in ecology is the Standardized Effect Size calculated
# as SES = (observations - mean(expectations)) / sd(expectations).
mean.null <- apply(null, 2, mean, na.rm = TRUE)
sd.null <- apply(null, 2, sd, na.rm = TRUE)
SES <- (obs - mean.null) / sd.null

plot(SES, ylim = range(c(-2, +2, SES)))
abline(h = c(-2, +2), lty = 2)

```

